

Class on 15/2/2019—3rd hr

White-Box testing:

White-box testing, sometimes called glass-box testing, is a test-case design philosophy that uses the control structure described as part of component-level design to derive test cases. Using white-box testing methods, you can derive test cases that

- (1) .guarantee that all independent paths within a module have been exercised at least once,
- (2) exercise all logical decisions on their true and false sides,
- (3) execute all loops at their boundaries and within their operational bounds, and
- (4) exercise internal data structures to ensure their validity

Black-Box testing:

Black-box testing, also called behavioral testing, focuses on the functional requirements of the software. That is, black-box testing techniques enable you to derive sets of input conditions that will fully exercise all functional requirements for a program. Black-box testing is not an alternative to white-box techniques. Rather, it is a complementary approach that is likely to uncover a different class of errors than whitebox methods.

Black-box testing attempts to find errors in the following categories:

- (1) incorrect or missing functions,
- (2) interface errors,
- (3) errors in data structures or external database access,
- (4) behavior or performance errors, and
- (5) initialization and termination errors.

Unlike white-box testing, which is performed early in the testing process, blackbox testing tends to be applied during later stages of testing (see Chapter 17). Because black-box testing purposely disregards control structure, attention is focused on the information domain. Tests are designed to answer the following questions:

How is functional validity tested?

- How are system behavior and performance tested?
- What classes of input will make good test cases?
- Is the system particularly sensitive to certain input values?
- How are the boundaries of a data class isolated?
- What data rates and data volume can the system tolerate?

- What effect will specific combinations of data have on system operation?

Differences between Black Box Testing vs White Box Testing

Software Testing can be majorly classified into two categories:

1. **Black Box Testing** is a software testing method in which the internal structure/ design/ implementation of the item being tested is NOT known to the tester
2. **White Box Testing** is a software testing method in which the internal structure/ design/ implementation of the item being tested is known to the tester.

Differences between Black Box Testing vs White Box Testing:

BLACK BOX TESTING	WHITE BOX TESTING
It is a way of software testing in which the internal structure or the program or the code is hidden and nothing is known about it.	It is a way of testing the software in which the tester has knowledge about the internal structure r the code or the program of the software.
It is mostly done by software testers.	It is mostly done by software developers.
No knowledge of implementation is needed.	Knowledge of implementation is required.
It can be referred as outer or external software testing.	It is the inner or the internal software testing.
It is functional test of the software.	It is structural test of the software.
This testing can be initiated on the basis of requirement specifications document.	This type of testing of software is started after detail design document.
No knowledge of programming is required.	It is mandatory to have knowledge of programming.
It is the behavior testing of the software.	It is the logic testing of the software.
It is applicable to the higher levels of testing of software.	It is generally applicable to the lower levels of software testing.
It is also called closed testing.	It is also called as clear box testing.
It is least time consuming.	It is most time consuming.

BLACK BOX TESTING	WHITE BOX TESTING
It is not suitable or preferred for algorithm testing.	It is suitable for algorithm testing.
Can be done by trial and error ways and methods.	Data domains along with inner or internal boundaries can be better tested.
Example: search something on google by using keywords	Example: by input to check and verify loops
Types of Black Box Testing: • A. Functional Testing • B. Non-functional testing • C. Regression Testing	Tyeps of White Box Testing: • A. Path Testing • B. Loop Testing • C. Condition testing

SYSTEM TESTING

System testing is actually a series of different tests whose primary purpose is to fully exercise the computer-based system. Although each test has a different purpose, all work to verify that system elements have been properly integrated and perform allocated functions. In the sections that follow, we discuss the types of system tests [BEI84] that are worthwhile for software-based systems.

- Recovery Testing
- Security Testing
- Stress Testing
- Performance Testing

Recovery Testing

Recovery testing is a system test that forces the software to fail in a variety of ways and verifies that recovery is properly performed. If recovery is automatic (performed by the system itself), reinitialization, checkpointing mechanisms, data recovery, and restart are evaluated for correctness. If recovery requires human intervention, the mean-time-to-repair (MTTR) is evaluated to determine whether it is within acceptable limits.

Security Testing

Security testing attempts to verify that protection mechanisms built into a system will, in fact, protect it from improper penetration. To quote Beizer [BEI84]: "The system's security must, of course, be tested for invulnerability from frontal attack—but must also be tested for invulnerability from flank or rear attack.

During security testing, the tester plays the role(s) of the individual who desires to penetrate the system. Anything goes! The tester may attempt to acquire passwords through external clerical means; may attack the system with custom software designed to breakdown any defenses that have been constructed; may overwhelm the system, thereby denying service to others; may purposely cause system errors, hoping to penetrate during recovery; may browse through insecure data, hoping to find the key to system entry

Stress Testing

Stress testing executes a system in a manner that demands resources in abnormal quantity, frequency, or volume. For example,

- (1) special tests may be designed that generate ten interrupts per second, when one or two is the average rate,
- (2) input data rates may be increased by an order of magnitude to determine how input functions will respond,
- (3) test cases that require maximum memory or other resources are executed,
- (4) test cases that may cause thrashing in a virtual operating system are designed,
- (5) test cases that may cause excessive hunting for disk-resident data are created. Essentially, the tester attempts to break the program.

Performance Testing

For real-time and embedded systems, software that provides required function but does not conform to performance requirements is unacceptable. Performance testing is designed to test the run-time performance of software within the context of an integrated system. Performance testing occurs throughout all steps in the testing process. Even at the unit level, the performance of an individual module may be assessed as white-box tests are conducted. However, it is not until all system elements are fully integrated that the true performance of a system can be ascertained. Performance tests are often coupled with stress testing and usually require both hardware and software instrumentation. That is, it is often necessary to measure resource utilization (e.g., processor cycles) in an exacting fashion. External instrumentation can monitor execution intervals, log events (e.g., interrupts) as they occur, and sample machine states on a regular basis. By instrumenting a system, the tester can uncover situations that lead to degradation and possible system failure.

Validation testing

Software validation is achieved through a series of black-box tests that demonstrate conformity with requirements. A test plan outlines the classes of tests to be conducted and a test procedure defines specific test cases that will be used to demonstrate conformity with requirements. Both the plan and procedure are designed to ensure that all functional requirements are satisfied, all behavioral characteristics are achieved, all performance requirements are attained, documentation is correct, and human-engineered and other requirements are met (e.g., transportability, compatibility, error recovery, maintainability)

Stages of Validation testing Process:

- **Validation Planning** – To plan all the activities that need to be included while testing.

- **Define Requirements** – To set goals and define the requirements for testing.
- **Selecting a Team** – To select a skilled and knowledgeable development team (the third party included).
- **Developing Documents** – To develop a user specification document describing the operating conditions.
- **Estimation/Evaluation** – To evaluate the software as per the specifications and submit a validation report.
- **Fixing bugs or Incorporating Changes** – To change the software so as to remove any errors found during evaluation.

Validation Testing Variations:

- **Component/Unit Testing** – The aim of the unit testing is to look for bugs in the software component. At the same time, it also verifies the work of modules and objects which can be tested separately.
- **Integration testing-** This is an important part of the software validation model, where the interaction between the different interfaces of the components is tested. Along with the interaction between the different parts of the system, the interaction of the system with the computer operating system, file system, hardware, and any other software system it might interact with, is also tested.
- **System testing-** System testing is carried out when the entire software system is ready. The main concern of system testing is to verify the system against the specified requirements. While carrying out the tests, the tester is not concerned with the internals of the system but checks if the system behaves as per expectations.
- **Acceptance testing-** During this testing, a tester literally has to think like the client and test the software with respect to user needs, requirements, business processes and determine whether the software can be handed over to the client or not.
- **Alpha testing-** This type of testing is done at the developers' site by potential customers/users. Any problems encountered during this testing are rectified by the developers then and there.
- **Beta testing-** Once the software passes the alpha testing stage, beta testing is done at the user's end.
- **Regression testing-** This testing is done after the desired changes or modifications are made to the existing code. The code, when put to test, may have certain errors that can be

resolved by making essential changes. The software is again put to test after these changes are made to check whether the new code fulfils customer requirements or not.

Testing is a set of activities that can be planned in advance and conducted systematically. For this reason a template for software testing—a set of steps into which you can place specific test case design techniques and testing methods—should be defined for the software process. A number of software testing strategies have been proposed in the literature. All provide you with a template for testing and all have the following generic characteristics:

- To perform effective testing, you should conduct effective technical reviews (Chapter 15). By doing this, many errors will be eliminated before testing commences.
 - Testing begins at the component level and works “outward” toward the integration of the entire computer-based system.
 - Different testing techniques are appropriate for different software engineering approaches and at different points in time.
 - Testing is conducted by the developer of the software and (for large projects) an independent test group.
 - Testing and debugging are different activities, but debugging must be accommodated in any testing strategy
1. Verification and Validation
 2. Organizing for Software Testing
 3. Software Testing Strategy—The Big Picture

Verification and Validation:

Software testing is one element of a broader topic that is often referred to as verification and validation (V&V). Verification refers to the set of tasks that ensure that software correctly implements a specific function. Validation refers to a different set of tasks that ensure that the software that has been built is traceable to customer requirements. Boehm [Boe81] states this another way:

Organizing for Software Testing

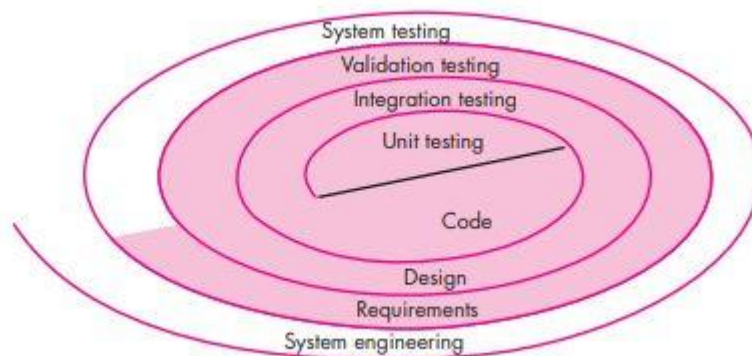
From a psychological point of view, software analysis and design (along with coding) are constructive tasks. The software engineer analyzes, models, and then creates a computer program and its documentation. Like any builder, the software

engineer is proud of the edifice that has been built and looks askance at anyone who attempts to tear it down. When testing commences, there is a subtle, yet definite, attempt to “break” the thing that the software engineer has built. From the point of view of the builder, testing can be considered to be (psychologically) destructive. So the builder treads lightly, designing and executing tests that will demonstrate that the program works, rather than uncovering errors. Unfortunately, errors will be present. And, if the software engineer doesn’t find them, the customer will! There are often a number of misconceptions that you might infer erroneously from the preceding discussion:

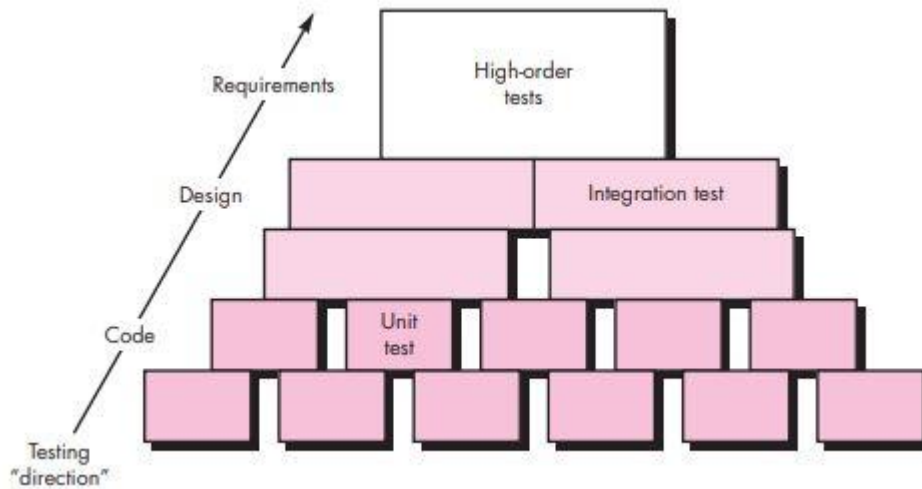
- that the developer of software should do no testing at all,
- that the software should be “tossed over the wall” to strangers who will test it mercilessly,
- that testers get involved with the project only when the testing steps are about to begin. Each of these statements is incorrect.

Software Testing Strategy—The Big Picture

Initially, system engineering defines the role of software and leads to software requirements analysis, where the information domain, function, behavior, performance, constraints, and validation criteria for software are established. Moving inward along the spiral, you come to design and finally to coding. To develop computer software, you spiral inward (counterclockwise) along streamlines that decrease the level of abstraction on each turn



Considering the process from a procedural point of view, testing within the context of software engineering is actually a series of four steps that are implemented sequentially. The steps are shown in Figure Initially, tests focus on each

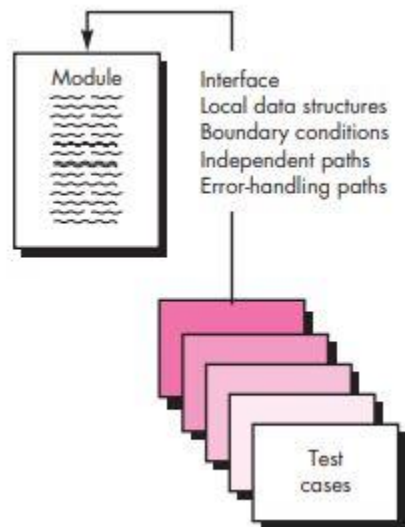


Test Strategies For Conventional Software

There are many strategies that can be used to test software. At one extreme, you can wait until the system is fully constructed and then conduct tests on the overall system in hopes of finding errors. This approach, although appealing, simply does not work. It will result in buggy software that disappoints all stakeholders. At the other extreme, you could conduct tests on a daily basis, whenever any part of the system is constructed. This approach, although less appealing to many, can be very effective. Unfortunately, some software developers hesitate to use it. What to do?

Unit Testing

Unit testing focuses verification effort on the smallest unit of software design—the software component or module. Using the component-level design description as a guide, important control paths are tested to uncover errors within the boundary of the module. The relative complexity of tests and the errors those tests uncover is limited by the constrained scope established for unit testing. The unit test focuses on the internal processing logic and data structures within the boundaries of a component. This type of testing can be conducted in parallel for multiple components.

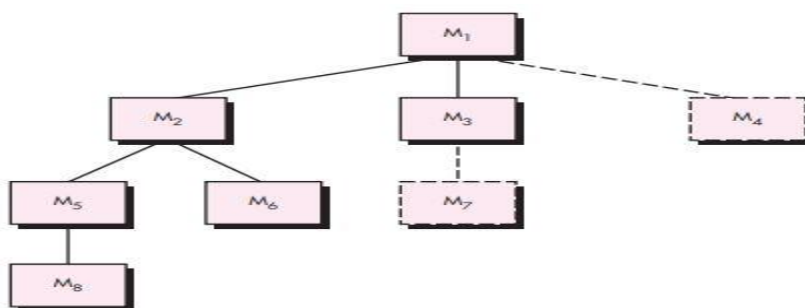


Integration Testing

A neophyte in the software world might ask a seemingly legitimate question once all modules have been unit tested: “If they all work individually, why do you doubt that they’ll work when we put them together?” The problem, of course, is “putting them together”—interfacing. Data can be lost across an interface; one component can have an inadvertent, adverse effect on another; subfunctions, when combined, may not produce the desired major function; individually acceptable imprecision may be magnified to unacceptable levels; global data structures can present problems. Sadly, the list goes on and on

Top-down integration.

Top-down integration testing is an incremental approach to construction of the software architecture. Modules are integrated by moving downward through the control hierarchy, beginning with the main control module



(main program). Modules subordinate (and ultimately subordinate) to the main control module are incorporated into the structure in either a depth-first or breadth-first manner.

Bottom-up integration.

Bottom-up integration testing, as its name implies, begins construction and testing with atomic modules (i.e., components at the lowest levels in the program structure). Because components are integrated from the bottom up, the functionality provided by components subordinate to a given level is always available and the need for stubs is eliminated. A bottom-up integration strategy may be implemented with the following steps:

- Low-level components are combined into clusters (sometimes called builds) that perform a specific software subfunction.
- A driver (a control program for testing) is written to coordinate test case input and output.
- The cluster is tested.
- Drivers are removed and clusters are combined moving upward in the program structure

Regression testing.

Each time a new module is added as part of integration testing, the software changes. New data flow paths are established, new I/O may occur, and new control logic is invoked. These changes may cause problems with functions that previously worked flawlessly. In the context of an integration test strategy, regression testing is the reexecution of some subset of tests that have already been conducted to ensure that changes have not propagated unintended side effects.

- A representative sample of tests that will exercise all software functions.
- Additional tests that focus on software functions that are likely to be affected by the change.
- Tests that focus on the software components that have been changed.

Smoke testing.

Smoke testing is an integration testing approach that is commonly used when product software is developed. It is designed as a pacing mechanism for time-critical projects, allowing the software team to assess the project on a frequent basis. In essence, the smoke-testing approach encompasses the following activities:

- Software components that have been translated into code are integrated into a build. A build includes all data files, libraries, reusable modules, and engineered components that are required to implement one or more product functions.
- A series of tests is designed to expose errors that will keep the build from properly performing its function. The intent should be to uncover “showstopper” errors that have the highest likelihood of throwing the software project behind schedule.
- The build is integrated with other builds, and the entire product (in its current form) is smoke tested daily. The integration approach may be top down or bottom up

